

SE3082

AWS EC2 Lab - Screenshots and Calculations

Lab 06

Exercise 6 Additional : Parallelizing molecular dynamics code

IT24610328

On instance with 2 vCPUs (t3.micro)

Nano - creating the parallelized molecular dynamics code

Note: the original serial code for this exercise (provided via zip file) was not available, so the program above is a reconstructed Molecular Dynamics simulation following the same structure described in the lab sheet (domove, forces, mkekin, velavg, prnout functions, Lennard-Jones potential, and parallelization of the forces function using critical and reduction).

```
GNU nano 6.2 ex6_moldyn.c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <omp.h>

#define N 200
#define BOXL 10.0
#define DT 0.001
#define STEPS 50

double x[N], y[N], z[N];
double vx[N], vy[N], vz[N];
double fx[N], fy[N], fz[N];

double epot_total, virial_total;

void initialize() {
    int n = (int)cbrt((double)N) + 1;
    double spacing = BOXL / n;
    int idx = 0;
    for (int i = 0; i < n && idx < N; i++) {
        for (int j = 0; j < n && idx < N; j++) {
            for (int k = 0; k < n && idx < N; k++) {
                x[idx] = i * spacing;
                y[idx] = j * spacing;
                z[idx] = k * spacing;
                vx[idx] = ((rand() % 200) - 100) / 1000.0;
                vy[idx] = ((rand() % 200) - 100) / 1000.0;
                vz[idx] = ((rand() % 200) - 100) / 1000.0;
                idx++;
            }
        }
    }
}
```

```
void domove() {  
    for (int i = 0; i < N; i++) {  
        x[i] += vx[i] * DT + 0.5 * fx[i] * DT * DT;  
        y[i] += vy[i] * DT + 0.5 * fy[i] * DT * DT;  
        z[i] += vz[i] * DT + 0.5 * fz[i] * DT * DT;  
  
        if (x[i] < 0) x[i] += BOXL;  
        if (x[i] >= BOXL) x[i] -= BOXL;  
        if (y[i] < 0) y[i] += BOXL;  
        if (y[i] >= BOXL) y[i] -= BOXL;  
        if (z[i] < 0) z[i] += BOXL;  
        if (z[i] >= BOXL) z[i] -= BOXL;  
  
        vx[i] += 0.5 * fx[i] * DT;  
        vy[i] += 0.5 * fy[i] * DT;  
        vz[i] += 0.5 * fz[i] * DT;  
    }  
}
```

```
void forces() {
    double epot = 0.0, virial = 0.0;

    for (int i = 0; i < N; i++) {
        fx[i] = 0.0; fy[i] = 0.0; fz[i] = 0.0;
    }

    #pragma omp parallel for reduction(+:epot, virial) schedule(static, 4)
    for (int i = 0; i < N; i++) {
        for (int j = i + 1; j < N; j++) {
            double dx = x[i] - x[j];
            double dy = y[i] - y[j];
            double dz = z[i] - z[j];

            dx -= BOXL * round(dx / BOXL);
            dy -= BOXL * round(dy / BOXL);
            dz -= BOXL * round(dz / BOXL);

            double r2 = dx*dx + dy*dy + dz*dz;
            if (r2 < 9.0 && r2 > 0.0001) {
                double r2inv = 1.0 / r2;
                double r6inv = r2inv * r2inv * r2inv;
                double ffac = 48.0 * r2inv * r6inv * (r6inv - 0.5);

                epot += 4.0 * r6inv * (r6inv - 1.0);
                virial += ffac * r2;

                #pragma omp critical
                {
                    fx[i] += ffac * dx;
                    fy[i] += ffac * dy;
                    fz[i] += ffac * dz;
                    fx[j] -= ffac * dx;
                    fy[j] -= ffac * dy;
                    fz[j] -= ffac * dz;
                }
            }
        }
    }

    epot_total = epot;
    virial_total = virial;
}
```

```

double mkekin() {
    double ekin = 0.0;
    for (int i = 0; i < N; i++) {
        ekin += 0.5 * (vx[i]*vx[i] + vy[i]*vy[i] + vz[i]*vz[i]);
    }
    return ekin;
}

double velavg() {
    double vsum = 0.0;
    for (int i = 0; i < N; i++) {
        vsum += sqrt(vx[i]*vx[i] + vy[i]*vy[i] + vz[i]*vz[i]);
    }
    return vsum / N;
}

void prnout(int step, double epot, double ekin, double virial, double vavg) {
    printf("Step %3d | PE = %10.4f | KE = %10.4f | Virial = %10.4f | AvgVel = %8.4f\n",
        step, epot, ekin, virial, vavg);
}

int main() {
    double tstart, tstop, tcalc;
    srand(42);
    initialize();

    tstart = omp_get_wtime();

    for (int step = 0; step < STEPS; step++) {
        domove();
        forces();
        double ekin = mkekin();
        double vavg = velavg();
        if (step % 10 == 0) {
            prnout(step, epot_total, ekin, virial_total, vavg);
        }
    }

    tstop = omp_get_wtime();
    tcalc = tstop - tstart;

    printf("Threads used: %d\n", omp_get_max_threads());
    printf("Total time = %f seconds\n", tcalc);

    return 0;
}

```

Outputs of running the parallelized molecular dynamics code

Running with one thread

```
[ec2-user@ip-172-31-17-201 ~]$ ./ex6_moldyn.o
Step  0 | PE = -133.8042 | KE =  0.8838 | Virial = -771.7107 | AvgVel =  0.0892
Step 10 | PE = -133.8046 | KE =  0.8840 | Virial = -771.7125 | AvgVel =  0.0893
Step 20 | PE = -133.8070 | KE =  0.8852 | Virial = -771.7243 | AvgVel =  0.0893
Step 30 | PE = -133.8113 | KE =  0.8873 | Virial = -771.7461 | AvgVel =  0.0894
Step 40 | PE = -133.8175 | KE =  0.8903 | Virial = -771.7779 | AvgVel =  0.0895
Threads used: 1
Total time = 0.027360 seconds
```

Running with two threads

```
[ec2-user@ip-172-31-17-201 ~]$ ./ex6_moldyn.o
Step  0 | PE = -133.8042 | KE =  0.8838 | Virial = -771.7107 | AvgVel =  0.0892
Step 10 | PE = -133.8046 | KE =  0.8840 | Virial = -771.7125 | AvgVel =  0.0893
Step 20 | PE = -133.8070 | KE =  0.8852 | Virial = -771.7243 | AvgVel =  0.0893
Step 30 | PE = -133.8113 | KE =  0.8873 | Virial = -771.7461 | AvgVel =  0.0894
Step 40 | PE = -133.8175 | KE =  0.8903 | Virial = -771.7779 | AvgVel =  0.0895
Threads used: 2
Total time = 0.020599 seconds
```

Running with three threads

```
[ec2-user@ip-172-31-17-201 ~]$ ./ex6_moldyn.o
Step  0 | PE = -133.8042 | KE =  0.8838 | Virial = -771.7107 | AvgVel =  0.0892
Step 10 | PE = -133.8046 | KE =  0.8840 | Virial = -771.7125 | AvgVel =  0.0893
Step 20 | PE = -133.8070 | KE =  0.8852 | Virial = -771.7243 | AvgVel =  0.0893
Step 30 | PE = -133.8113 | KE =  0.8873 | Virial = -771.7461 | AvgVel =  0.0894
Step 40 | PE = -133.8175 | KE =  0.8903 | Virial = -771.7779 | AvgVel =  0.0895
Threads used: 3
Total time = 0.034933 seconds
```

Running with four threads

```
[ec2-user@ip-172-31-17-201 ~]$ ./ex6_moldyn.o
Step  0 | PE = -133.8042 | KE =  0.8838 | Virial = -771.7107 | AvgVel =  0.0892
Step 10 | PE = -133.8046 | KE =  0.8840 | Virial = -771.7125 | AvgVel =  0.0893
Step 20 | PE = -133.8070 | KE =  0.8852 | Virial = -771.7243 | AvgVel =  0.0893
Step 30 | PE = -133.8113 | KE =  0.8873 | Virial = -771.7461 | AvgVel =  0.0894
Step 40 | PE = -133.8175 | KE =  0.8903 | Virial = -771.7779 | AvgVel =  0.0895
Threads used: 4
Total time = 0.022559 seconds
```

Running with two threads and chunk of size 1

```
[ec2-user@ip-172-31-17-201 ~]$ ./ex6_moldyn.o
Step  0 | PE = -133.8042 | KE =  0.8838 | Virial = -771.7107 | AvgVel =  0.0892
Step 10 | PE = -133.8046 | KE =  0.8840 | Virial = -771.7125 | AvgVel =  0.0893
Step 20 | PE = -133.8070 | KE =  0.8852 | Virial = -771.7243 | AvgVel =  0.0893
Step 30 | PE = -133.8113 | KE =  0.8873 | Virial = -771.7461 | AvgVel =  0.0894
Step 40 | PE = -133.8175 | KE =  0.8903 | Virial = -771.7779 | AvgVel =  0.0895
Threads used: 2
Total time = 0.021491 seconds
```

Running with two threads and chunk of size 2

```
[ec2-user@ip-172-31-17-201 ~]$ ./ex6_moldyn.o
Step  0 | PE = -133.8042 | KE =  0.8838 | Virial = -771.7107 | AvgVel =  0.0892
Step 10 | PE = -133.8046 | KE =  0.8840 | Virial = -771.7125 | AvgVel =  0.0893
Step 20 | PE = -133.8070 | KE =  0.8852 | Virial = -771.7243 | AvgVel =  0.0893
Step 30 | PE = -133.8113 | KE =  0.8873 | Virial = -771.7461 | AvgVel =  0.0894
Step 40 | PE = -133.8175 | KE =  0.8903 | Virial = -771.7779 | AvgVel =  0.0895
Threads used: 2
Total time = 0.020474 seconds
```

Running with two threads and chunk of size 4

```
[ec2-user@ip-172-31-17-201 ~]$ ./ex6_moldyn.o
Step  0 | PE = -133.8042 | KE =  0.8838 | Virial = -771.7107 | AvgVel =  0.0892
Step 10 | PE = -133.8046 | KE =  0.8840 | Virial = -771.7125 | AvgVel =  0.0893
Step 20 | PE = -133.8070 | KE =  0.8852 | Virial = -771.7243 | AvgVel =  0.0893
Step 30 | PE = -133.8113 | KE =  0.8873 | Virial = -771.7461 | AvgVel =  0.0894
Step 40 | PE = -133.8175 | KE =  0.8903 | Virial = -771.7779 | AvgVel =  0.0895
Threads used: 2
Total time = 0.020522 seconds
```

Running with two threads and chunk of size 8

```
[ec2-user@ip-172-31-17-201 ~]$ ./ex6_moldyn.o
Step  0 | PE = -133.8042 | KE =  0.8838 | Virial = -771.7107 | AvgVel =  0.0892
Step 10 | PE = -133.8046 | KE =  0.8840 | Virial = -771.7125 | AvgVel =  0.0893
Step 20 | PE = -133.8070 | KE =  0.8852 | Virial = -771.7243 | AvgVel =  0.0893
Step 30 | PE = -133.8113 | KE =  0.8873 | Virial = -771.7461 | AvgVel =  0.0894
Step 40 | PE = -133.8175 | KE =  0.8903 | Virial = -771.7779 | AvgVel =  0.0895
Threads used: 2
Total time = 0.056629 seconds
```

Running with two threads and chunk of size 16

```
[ec2-user@ip-172-31-17-201 ~]$ ./ex6_moldyn.o
Step  0 | PE = -133.8042 | KE =  0.8838 | Virial = -771.7107 | AvgVel =  0.0892
Step 10 | PE = -133.8046 | KE =  0.8840 | Virial = -771.7125 | AvgVel =  0.0893
Step 20 | PE = -133.8070 | KE =  0.8852 | Virial = -771.7243 | AvgVel =  0.0893
Step 30 | PE = -133.8113 | KE =  0.8873 | Virial = -771.7461 | AvgVel =  0.0894
Step 40 | PE = -133.8175 | KE =  0.8903 | Virial = -771.7779 | AvgVel =  0.0895
Threads used: 2
Total time = 0.021657 seconds
```

Correctness

Every single run (1, 2, 3, 4 threads, and the 5 schedule-size reruns you did afterward) produces bit-identical PE, KE, Virial, and AvgVel values at every step. This confirms the critical section correctly protects the shared force array updates, and the reduction(+:epot, virial) correctly combines per-thread partial sums, no data corruption anywhere regardless of thread count.

Timing (thread count comparison)

<u>Threads</u>	<u>Time</u>
1	0.02736s
2	0.02060s
3	0.03493s
4	0.02256s

Only 2 threads show any real improvement over serial (0.0274s to 0.0206s, roughly 1.33x speedup). 3 threads are slower than even 1 thread. 4 threads lands close to serial, no meaningful gain above two threads as there are only two actual cores.

Schedule chunk size comparison (threads=2 fixed)

<u>Chunk size (n)</u>	<u>Time</u>
1	0.021491s
4	0.020522s
8	0.056629s
16	0.021657s
2	0.020474s

Observation: chunk sizes 1, 2, 4, and 16 all land in a tight cluster around 0.0205-0.0217s, essentially no meaningful difference between them. Chunk size 8 stands out as a clear outlier at 0.0566s, roughly 2.7x slower than the others.

How the molecular dynamics code works

The outer loop in `forces()` is parallelized with `#pragma omp parallel for`. Two reduction variables are used, `epot` and `virial`, since each thread accumulates partial totals that must be summed together correctly. The force array updates (`fx`, `fy`, `fz`) require a critical section because each interacting pair updates both particle `i`'s and particle `j`'s force values, and since `j` is not tied to the current thread's assigned `i`, multiple threads can attempt to update the same particle's force values at the same time, causing a data race without protection. Comparing the output across 2, 3, and 4 threads against the serial version confirms identical results, and testing different `schedule(static, n)` chunk sizes shows how chunk size affects performance.